ENTERPRISE SALES DATA WAREHOUSE

Technical Project Report

SQL Server | SSIS | Python | Dimensional Modelling | Power BI

Prepared for: Data Engineering Final Capstone
Prepared: 31 October 2026

Document Scope and Evidence Note

This report is based on the supplied project files, screenshots and previously reported execution status. It distinguishes implemented source artifacts from results requiring fresh validation. Financial figures are reproduced as displayed in the supplied Power BI screenshots and are not independently audited.

Contents

1. Executive Summary

2. Business Problem and Objectives

3. Scope and Technology Stack

4. Source Data and Volume

5. Architecture and Data Flow

6. Python Data Profiling

7. Warehouse Model

8. ETL and Automation

9. Power BI Reporting and Interpretation

10. Data Quality and Validation

11. Challenges and Decisions

12. Limitations and Future Improvements

13. Conclusion

1. Executive Summary

This project builds a data engineering and analytics workflow for a multi-channel retail business. It brings sales, returns, targets, customer, product, store, employee and product-price-history data into a SQL Server warehouse and exposes business-oriented summaries through Power BI.

The solution combines SQL Server source/staging/warehouse databases, SSIS packages for ETL orchestration, a reusable Python/Pandas profiler, dimensional modelling, Customer SCD Type 2 processing and Power BI reporting. A deployed Master ETL package was reported to execute successfully through SQL Server Agent. Power BI Service scheduled refresh remains pending until source credentials and gateway connectivity are configured and tested.

2. Business Problem and Objectives

The business scenario describes a Nigerian multi-channel retailer whose operational information is distributed across SQL tables, CSV extracts and an Excel sales-target file. Management needs consistent views of sales, profitability, customers, products, stores, employees, returns and targets.

The project objectives are to ingest the source datasets, separate ingestion from analytics using a staging layer, profile data quality, load a dimensional warehouse, preserve customer history, support incremental processing, validate warehouse data and publish a Power BI dashboard.

3. Scope and Technology Stack

Technologies present in the project are SQL Server 2025/T-SQL, SQL Server Integration Services (SSIS), Python with pandas and pyodbc, Power BI Desktop/Service, SSISDB and SQL Server Agent. The repository includes SQL scripts, SSIS package source, Python scripts, source extracts, a PBIX report, a data dictionary and validation scripts.

The repository is a reproducible project snapshot rather than a one-click installer: SQL Server names, credentials, SSIS connection managers, the Python executable and script paths must be configured for each environment.

4. Source Data and Volume

The project uses eight datasets: customers, products, stores, employees, sales transactions, returns, sales targets and product price history. The provided source files contain 25,000 customers, 5,000 products, 250 stores, 1,200 employees,

300,150 sales rows, 12,000 returns, 2,500 target rows and 2,978 product-price-history rows.

The source sales file includes intentional quality challenges. Verification during repository preparation found 120 sales rows with a missing CustomerID, 100 invalid NetAmount values when converted numerically, and 150 exact duplicate sales rows. These counts are evidence from the provided CSVs and should be reconciled against any loaded database copy before final submission.

Dataset	Rows
Customers	25,000
Products	5,000
Stores	250
Employees	1,200
Sales transactions	300,150
Returns	12,000
Sales targets	2,500
Product price history	2,978

5. Architecture and Data Flow

The high-level flow is: source CSV/Excel and SQL tables → source database (src) → SSIS staging load → staging database (stg) → Python data profiling → warehouse processing → warehouse dimensions and facts (dw) → Power BI semantic model and report.

The staging layer isolates ingestion from dimensional modelling and provides a place to inspect raw values, track load timestamps and preserve rejected values where reject outputs are implemented. The warehouse organizes descriptive attributes in dimensions and measurable business events in fact tables.

6. Python Data Profiling

The Python orchestration script iterates over eight approved staging tables and launches the table-level profiler as a subprocess. A non-zero subprocess return code stops the overall profiling run. The profiler reports row and column counts, column types, missing values, exact duplicate rows, business-key duplicates, distinct counts, categorical distributions, numeric summaries and date ranges. Customer checks also inspect age ranges and the order of signup/activity dates.

The profiler is a diagnostic component: it reports quality signals but does not by itself clean every problem. Actual cleaning, standardization, rejection or remediation must be verified in the SSIS and SQL transformations. The public repository copy also safely coerces Age to numeric before applying range checks, since staging values may arrive as text.

7. Warehouse Model

The warehouse schema defines five dimensions: DimDate, DimCustomer, DimProduct, DimStore and DimEmployee. It defines three facts: FactSales, FactReturns and FactSalesTarget. Surrogate dimension keys support analytical joins, while business identifiers remain available for traceability.

The declared FactSales grain is one row per source sales line, identified by OrderID plus SourceRecordID. FactReturns is intended to contain one row per ReturnID. FactSalesTarget is intended to contain one row per TargetDate, StoreID and Category. The grain matters: an OrderID can legitimately appear on multiple sales lines, so OrderID-only duplicate checks are diagnostics rather than proof of duplicate facts.

The warehouse SQL includes Customer Type 2 processing to preserve historical versions when tracked customer attributes change. Staging SQL includes watermark and delta-merge procedure definitions. The extent to which every delta procedure is connected to the scheduled orchestration should be validated in the target environment.

8. ETL and Automation

The SSIS project includes staging-load packages, a warehouse-load package and a Master ETL package. The master package coordinates staging, Python profiling and warehouse processing. The project owner reported a successful execution of the deployed Master ETL package through SQL Server Agent, with the run visible in SSISDB execution reporting.

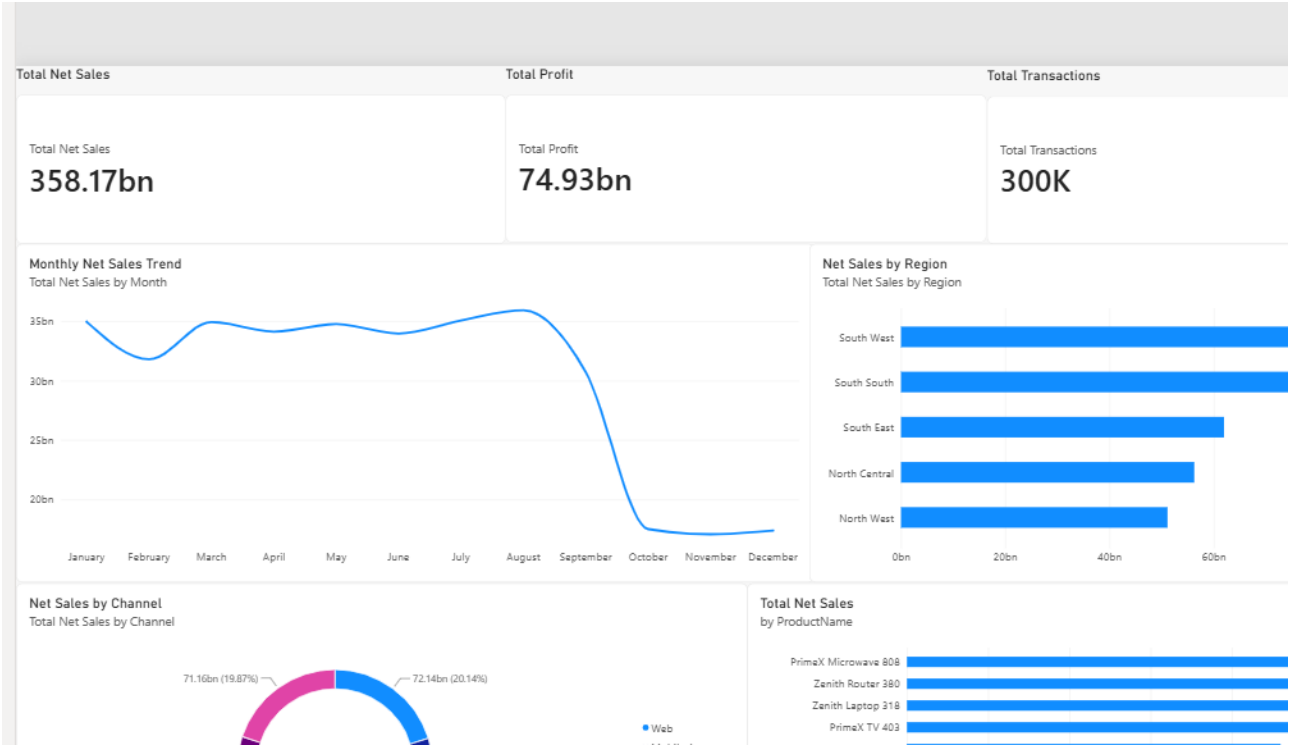
SQL Server Agent automation and Power BI refresh automation are separate. Agent execution updates the warehouse. Power BI Service must then refresh its semantic model to retrieve updated data. The latest provided Power BI settings screenshot showed scheduled refresh disabled because at least one data source credential was missing. A gateway/source mapping, credentials, manual refresh test and later scheduled refresh test remain necessary before claiming end-to-end cloud refresh is operational.

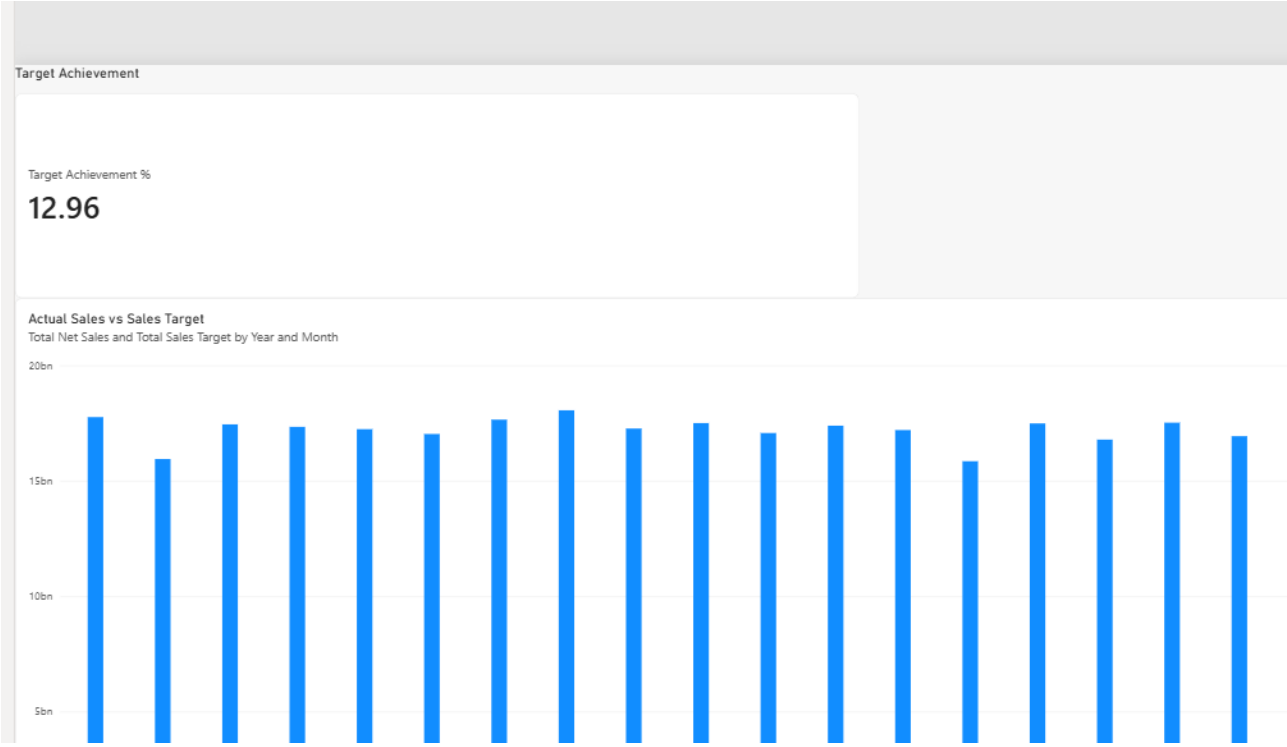
9. Power BI Reporting and Interpretation

The report has two pages. Page 1 contains KPI cards for net sales, profit and transactions; a monthly net sales trend; net sales by region; net sales by channel; and product sales ranking. Page 2 contains a target-achievement KPI and a monthly actual-sales-versus-target chart.

The supplied dashboard screenshot displays total net sales of 358.17bn, total profit of 74.93bn and total transactions of 300K. These values are reproduced as displayed in the report; the screenshots do not independently establish currency, financial unit correctness or the exact DAX definitions. The channel visual shows approximately even contributions across five channels, while the region visual ranks South West highest among the shown regions. Product sales are ranked by net sales, not profitability.

Two interpretation checks are important before presenting business conclusions. First, the monthly trend drops sharply in the later displayed months; the source date range previously reported ends on 23 September 2026, so partial or missing periods may explain part of the shape. Second, the Target Achievement KPI shows 12.96 while the actual-versus-target bars visually suggest actual sales greatly exceed targets. The DAX formula, percentage formatting, target aggregation, relationship paths and date/filter context must be checked so the two visuals reconcile.





10. Data Quality and Validation

The repository includes a starter SQL quality suite that checks missing customer keys in FactSales, customer current-row rules, orphan customer keys, profit arithmetic and duplicate OrderID diagnostics. It explicitly notes that additional freshness, source/target count, reconciliation, valid-date and required-field checks remain to be added.

The source dataset has known missing identifiers, invalid numeric values and duplicate rows. The project includes rejected-row structures in staging SQL, including a rejected-sales structure. The report does not claim that every quality issue is automatically cleaned or rejected. Validation should compare source/staging/warehouse row counts, test referential integrity, verify financial arithmetic and retain the output from repeat ETL runs.

11. Challenges and Decisions

The project addressed the need to separate raw ingestion from warehouse modelling, profile multiple tables with a reusable Python entry point, preserve customer history through SCD Type 2, handle invalid numeric sales values through rejection-oriented staging structures, and automate package execution through SQL Server Agent.

Important design decisions include defining business keys for each table, documenting fact grain, retaining source record identifiers for traceability, and separating warehouse automation from Power BI refresh. These choices make the flow easier to validate and explain.

12. Limitations and Future Improvements

Priority next steps are: complete gateway and data-source credential setup for Power BI Service refresh; reconcile the target achievement KPI with the target comparison chart; verify late-period date completeness; run the full SQL quality suite and capture results; and retain SSISDB/Agent success evidence.

Further enhancements can include a persisted quality-results log, expanded referential-integrity and financial reconciliation checks, measured before/after query tuning with execution plans, index analysis, more parameterized SSIS configuration, and a documented clean-environment deployment procedure. These are recommendations and should not be presented as completed work unless implemented and tested.

13. Conclusion

The project demonstrates an integrated data engineering workflow using SQL Server, SSIS, Python profiling, dimensional modelling, incremental/SCD processing and Power BI. It creates a structured path from source data to analytical reporting and establishes a basis for repeatable ETL and business analysis.

The strongest final submission will pair the code and documentation with evidence: successful Agent/SSISDB execution, warehouse row counts and quality checks, SCD Type 2 test results, dashboard screenshots, and a successful Power BI refresh history entry once gateway credentials are configured.